

Introduction and System Overview

Welcome to Power-Rangers

Welcome to the User Manual for the **Delhi Power Demand Predictor**. This system is an end-to-end, machine learning-driven platform designed to predict electricity demand for the Delhi region. It automates the ingestion of weather and load data, trains a Temporal Fusion Transformer (TFT) model, and serves probabilistic forecasts (P10, P50, P90) through an interactive web dashboard.

1.1 Intended Audience

This manual is intended for:

- **Data Scientists & ML Engineers:** Managing data pipelines, model training, and performance evaluation.
- **System Operators (SLDC):** Utilizing the frontend dashboard to monitor power demand forecasts and peak detection.
- **Software Developers:** Setting up, configuring, and maintaining the application infrastructure.

1.2 Key Features

- **Automated Data Pipelines:** Scrapes historical load data directly from SLDC and fetches weather telemetry via API.
- **Probabilistic Forecasting:** Generates 48-hour recursive forecasts predicting base (P50), optimistic (P10), and conservative (P90) energy demand.
- **Resilient Training:** Features robust checkpoint management to pause and resume model training without data loss.
- **High-Performance Dashboard:** A responsive React/Vite web interface for visualizing trends, anomalies, and extreme load peaks.

1.3 System Architecture at a Glance

The system is divided into two primary subsystems:

1. **Backend (Python):** Handles Data Acquisition (`src/ingestion`), Feature Engineering (`src/pipelines`), Model Training (`src/training`), and Inference (`src/forecast`)
2. **Frontend (TypeScript/React):** An interactive UI (`frontend/src`) serving visualizations using TailwindCSS and Aceternity UI components.

Installation and Setup

1.1 Prerequisites

Before installing the application, ensure your environment meets the following requirements:

- **Operating System:** Linux, macOS, or Windows (WSL recommended).
- **Python:** Version 3.9 to 3.11.
- **Node.js & npm:** Node version 18.x or higher.
- **Hardware:** A GPU with at least 6GB VRAM is highly recommended for model training.

1.2 Backend Setup (Data & Modeling)

1. Clone the Repository:

```
git clone https://github.com/Aman162004/Power-Rangers.git cd
Power-Rangers
```

2. Create a Virtual Environment:

```
python -m venv venv
source venv/bin/activate # On Windows use: venv\Scripts ctivate
```

3. Install Python Dependencies:

```
pip install -r requirements.txt
```

4. Configure Environment Variables:

Use the `config/config.yaml` file to set your API keys for the weather
fetcher and
configure database paths.

1.3 Frontend Setup (User Dashboard)

1. **Navigate to the Frontend Directory:**

```
cd frontend
```

2. **Install Node Modules:**

```
npm install
```

3. **Verify Environment Files:** Check the vite-env.d.ts and vite.config.ts to ensure API endpoints point to your local Python backend (usually `http://localhost:8000`).

Page 3: Data Pipeline & Model Training

3.1 Running the Data Ingestion Pipeline

The Data Acquisition system automatically collects load and weather data. To execute the ingestion pipeline, run:

```
python src/pipelines/main_ingestion.py
```

What happens?

- The `LoadFetcher` scrapes the SLDC portal.
- The `WeatherFetcher` reaches out to the configured weather API.
- Raw data is saved to `data/raw/` and logged in the `ingestion_manifest.json`.

3.2 Feature Engineering

Once raw data is available, it must be cleaned, interpolated, and engineered (adding temporal and holiday features) for the model.

```
python src/pipelines/prepare_historical_data.py
```

Output: The final clean datasets are stored in

`data/historical/final_processed/` along with

a `data_quality_report.json`.

3.3 Training the TFT Model

To train a new Temporal Fusion Transformer model on the processed historical data:

```
python src/training/training_pipeline.py
```

Checkpoint Management:

The system saves training checkpoints automatically inside the `models/runs/` directory (e.g., `epoch=01-val_loss=96.36.ckpt`). If training is interrupted, the `run_manager.py` will automatically resume from the `last.ckpt` file natively protecting against data loss.

Page 4: Forecasting and Inference

4.1 Understanding the Inference Output

The core value of the Power-Rangers system is its ability to forecast demand. The model does not output a single number; rather, it outputs a probabilistic distribution to help grid operators manage risk:

- **P50 (Median):** The most likely demand forecast.
- **P90 (Upper Bound):** A conservative high-end estimate (e.g., unexpected heatwave).
- **P10 (Lower Bound):** A low-end estimate.

4.2 Running a Manual Forecast

To generate a 48-hour forward-looking forecast from the command line:

```
python src/forecast/main_inference.py
```

This script will:

1. Load the best-performing model from `models/final_model/`.
2. Ingest the last known context window (historical lookahead).
3. Eject the recursive 48-hour output into the `src/shared/artifact_repository.py`.

4.3 Peak Detection and Evaluation

To specifically analyze peaks or evaluate model accuracy against a holdout test set:

- **Peak Detection:** `python src/forecast/peak_detection.py` (Flags anomalies and upcoming energy spikes).
- **Evaluation:** `python src/evaluation/evaluation.py` (Exports accuracy metrics).

Page 5: Frontend Dashboard &

Troubleshooting

5.1 Launching the Web Dashboard

System operators interact with the forecasts primarily through the React web interface.

1. **Start the Python Backend API** (from the root directory):

```
python backend/main.py
```

2. **Start the Frontend Development Server** (from the `frontend` directory):

```
npm run dev
```

3. **Open your browser** and navigate to `http://localhost:5173`.

Here, you can view the Demand Forecast Charts, Weather Overlays, and Peak Load Alerts.

5.2 Troubleshooting & FAQ

Q: I am getting a `CUDA Out of Memory` error during training.

- **Cause:** The model batch size exceeds your GPU hardware limits.
- **Fix:** Reduce the `batch_size` parameter in `config/config.yaml`. Ensure you have at least 6GB of VRAM available as per the system requirements.

Q: The SLDC Data Scraper is failing or timing out.

- **Cause:** The SLDC portal may be rate-limiting your IP, or their server is down.
- **Fix:** The system uses an automated `NFR-SAFE-2` fallback (read the last known good cached state. Check fallback data is present. `data/historical/raw/`). It will automatically attempt to e offline

Q: Node modules are failing to install (`npm ERR!`

).

- **Cause:** Incompatible Node versions or missing permissions.

- **Fix:** Ensure you are using Node 18+. Try deleting the `package-lock.json` and running `npm cache clean --force` followed by `npm install`.